

Aplicație de tip Internet Archive

Razvan, grupa C112-C, ATM

Ce trebuie să fac

Tema e să construiesc ceva care salvează o pagină web (sau un domeniu întreg) astfel încât să o pot deschide mai târziu offline, exact cum arăta atunci când am salvat-o. Practic un mini Wayback Machine.

Pentru o pagină simplă, trebuie să salvez HTML-ul și tot ce ține de el (poze, css, js), și să schimb linkurile din pagină ca să poarte spre fișierele descărcate local, nu spre internet. Pentru un domeniu întreg, fac același lucru dar pentru toate paginile la care ajung din acel domeniu, iar paginile alea trebuie găsite fie urmărind linkurile din pagină în pagină, fie încercând o listă de path-uri des întâlnite (/about, /profile, /index, lucruri de genul ăsta) care poate nu apar legate nicăieri direct.

Nu am voie să folosesc wget --mirror sau ceva similar care rezolvă totul automat, trebuie să scriu eu logica de crawling și de înlocuire a linkurilor.

Bash vs Python

Inițial m-am gândit s-o fac în Bash pur, cu curl + grep + sed, dar partea de parsare HTML cu regex e un coșmar (HTML nu e un limbaj care se pretează la regex, iar sed strică fișierul dacă nu scapi caracterele speciale din URL-uri). Îmi permit și Python, așa că merg pe varianta asta, e mult mai puțin fragilă:

- requests în loc de curl, pentru descărcat
- BeautifulSoup ca să parsez HTML-ul corect (găsește el tag-urile img, link, script, cu tot cu attribute), nu mai caut eu manual cu regex
- urllib.parse.urljoin ca să rezolv URL-urile relative față de cele absolute, ceva ce în Bash trebuia să fac manual și probabil greșit pe alocuri
 - un set() normal pentru URL-uri vizitate și un deque pentru coadă, în loc de fișiere text care simulează structuri de date

Cum arată, mai concret

Am un fetch simplu:

```
import requests

def fetch(url, timeout=10):
    resp = requests.get(url, timeout=timeout)
    return resp.status_code, resp.content
except requests.RequestException:
    return None, None
```

Din HTML scot resursele cu BeautifulSoup, nu cu regex:

```
from bs4 import BeautifulSoup
from urllib.parse import urljoin

def gaseste_resurse(html, base_url):
    soup = BeautifulSoup(html, "html.parser")
    resurse = []
    for tag, attr in [("img", "src"), ("script", "src"), ("link", "href")]:
        for el in soup.find_all(tag):
            val = el.get(attr)
            if val:
                resurse.append(urljoin(base_url, val))
    return resurse, soup
```

Pentru CSS mai am nevoie de un regex simplu care caută url(...), pentru că BeautifulSoup nu parsează CSS:

```
import re

def gaseste_urls_css(css_text, base_url):
    urls = re.findall(r'url\(((["\']?)(.*?)\1\)', css_text)
    return [urljoin(base_url, u[1]) for u in urls]
```

Partea de rescriere a linkurilor e mult mai simplă acum, pentru că modific direct atributul din obiectul soup, numai umblu cu sed și escaping:

```
def rescrie_linkuri(soup, url_to_local):
    for tag, attr in [("img", "src"), ("script", "src"), ("link", "href")]:
        for el in soup.find_all(tag):
            val = el.get(attr)
            if val in url_to_local:
                el[attr] = url_to_local[val]
    return str(soup)
```

Pentru crawling, coada și setul de vizitate sunt doar structuri Python obișnuite:

```
from collections import deque
from urllib.parse import urlparse
```

```
def crawleaza(start_url, max_depth=2):

    vizitate = set([start_url])
    coada = deque([(start_url, 0)])
    domeniu = urlparse(start_url).netloc

    while coada:
        url, depth = coada.popleft()
        status, html = fetch(url)
        if status != 200:
            continue
        # ... procesare pagina (salvare, extragere resurse) ...
        if depth < max_depth:
            soup = BeautifulSoup(html, "html.parser")
            for a in soup.find_all("a", href=True):
                link = urljoin(url, a["href"])
                if urlparse(link).netloc == domeniu and link not in vizitate:
                    vizitate.add(link)
                    coada.append((link, depth + 1))
```

Iar pentru lista de path-uri comune, doar dau requests.get pe fiecare și văd ce răspunde cu 200:

```
wordlist = ["/index", "/about", "/profile", "/files", "/login", "/contact"]
```

```
def probeaza_wordlist(base_url):
    gasite = []
    for path in wordlist:
        url = base_url.rstrip("/") + path
        status, _ = fetch(url)
        if status == 200:
            gasite.append(url)
    return gasite
```

Pe scurt

Ideea de bază rămâne aceeași indiferent de limbaj: descarci o pagină, îi găsești resursele, le aduci local, rescrii linkurile, și dacă vrei un domeniu întreg mai adaugi o coadă de crawling și o listă de path-uri de încercat pe lângă linkurile găsite normal. Diferența e că în Python nu mai trebuie să lupt cu regex-uri pentru HTML sau cu escaping în sed, biblioteci ca BeautifulSoup fac partea grea, iar mie îmi rămâne să mă ocup de logica de crawling și de partea de organizare a fișierelor.